

HƯỚNG DẪN QUY TRÌNH PHÁT TRIỂN & HỢP TÁC SPRINT 2

Tài liệu này hướng dẫn chi tiết về **quy trình hợp tác phát triển mới đã được rút gọn** của dự án. Tất cả các thành viên trong đội ngũ phát triển (Backend .NET, Backend Spring, Frontend) cần đọc kỹ và tuân thủ các quy tắc dưới đây trong suốt Sprint 2.

1. BỐI CẢNH & THAY ĐỔI CỐT LÕI

Để đẩy nhanh tiến độ và loại bỏ các thủ tục rườm rà, chúng ta đã **tối giản hóa quy trình Git và quản lý Issue**:

- 1. Gộp Issue:** Đóng toàn bộ các task nhỏ nhất. Thay vào đó, mỗi mảng tính năng lớn chỉ có duy nhất **1 issue lớn** đại diện trên GitHub.
- 2. Tự do đặt tên & Commit:** Không còn bắt buộc tiền tố tên nhánh (`feature/` , `fix/` ...) hay quy chuẩn tin nhắn commit nghiêm ngặt (Conventional Commits).
- 3. Bảo vệ nhánh cốt lõi:** Nhánh `dev` và `main` được bảo vệ bằng Git Hooks, ngăn chặn tuyệt đối việc đẩy code trực tiếp (`git push`).

2. QUY TRÌNH GIT & PHÂN NHÁNH MỚI

Tất cả lập trình viên bắt buộc phải tuân theo luồng làm việc 3 bước sau:

```
[Nhánh dev sạch] → [Tạo nhánh riêng] → [Commit tự do & Push] → [Tạo Pull Request]
```

✦ Bước 1: Tạo nhánh làm việc riêng từ `dev`

Trước khi code, hãy cập nhật code mới nhất từ nhánh chung và tạo một nhánh riêng tuân thủ đúng quy chuẩn sau:

- **Quy chuẩn đặt tên nhánh:** `<github-username>/<issue-id>-<short-name>`
- Trong đó:
- `<github-username>` : Tên tài khoản GitHub của bạn (ví dụ: `Ahug05` , `ToTrieuTien` , `fong-gif`).
- `<issue-id>` : Số thứ tự Issue trên GitHub (ví dụ: `71` , `72` , `73` , `74`).
- `<short-name>` : Tên ngắn gọn mô tả tính năng bằng tiếng Anh, viết thường, phân tách bằng dấu gạch ngang (ví dụ: `users-crud` , `manage-floors`).

Ví dụ thực tế cho từng thành viên:

- **Ahug05** (Làm CRUD Users - Issue #71):

```
git switch dev
git pull origin dev
git switch -c Ahug05/71-users-crud
```

- **ToTrieuTien** (Làm CRUD Cấu trúc bãi xe - Issue #72):

```
git switch dev
git pull origin dev
git switch -c ToTrieuTien/72-manage-floors
```

- **fong-gif** (Làm API chỉ đọc & Dashboard - Issue #73):

```
git switch dev
git pull origin dev
git switch -c fong-gif/73-dashboard-api
```

- **Frontend Dev** (Làm UI Đăng nhập & Quản trị - Issue #74):

```
git switch dev
git pull origin dev
git switch -c <your-github-username>/74-login-ui
```

✦ Bước 2: Lập trình, Commit và Push

Bạn có thể commit bất kỳ lúc nào với các tin nhắn mô tả công việc mình đang làm. Sau đó, hãy đẩy nhánh của bạn lên remote:

Ví dụ thực tế cho từng thành viên:

- **Ahug05:**

```
git add .
git commit -m "hoàn thành thiết lập database entity cho users"
git push origin Ahug05/71-users-crud
```

- **ToTrieuTien:**

```
git add .
git commit -m "thêm api crud cho floor và area"
git push origin ToTrieuTien/72-manage-floors
```

- **fong-gif:**

```
git add .
git commit -m "viết api thống kê slots bận và trống"
git push origin fong-gif/73-dashboard-api
```

- **Frontend Dev:**

```
git add .  
git commit -m "thiết kế giao diện đăng nhập và route guard bảo vệ"  
git push origin <your-github-username>/74-login-ui
```

✦ Bước 3: Tạo Pull Request (PR) để hợp nhất vào `dev`

Lên giao diện GitHub, tạo một **Pull Request** hướng về nhánh `dev` để kiểm tra và hợp nhất code.

Ví dụ thực tế cho từng thành viên:

- **Ahug05:** Tạo Pull Request từ nhánh `Ahug05/71-users-crud` cần merge vào nhánh `dev`.
- **ToTrieuTien:** Tạo Pull Request từ nhánh `ToTrieuTien/72-manage-floors` cần merge vào nhánh `dev`.
- **fong-gif:** Tạo Pull Request từ nhánh `fong-gif/73-dashboard-api` cần merge vào nhánh `dev`.
- **Frontend Dev:** Tạo Pull Request từ nhánh `<your-github-username>/74-login-ui` cần merge vào nhánh `dev`.

Lưu ý

⚠ **Chú ý:** Lệnh `git push origin dev` sẽ bị chặn trực tiếp bởi Git Hook để bảo vệ mã nguồn chung. Bạn chỉ có thể đưa code vào `dev` bằng Pull Request.

💡 3. NGUYÊN TẮC "PULL REQUEST NHỎ - MERGE SỚM"

Mặc dù cả Sprint 2 bạn chỉ có 1 Issue lớn, nhưng **đừng đợi đến cuối Sprint mới tạo PR**. Việc ôm code quá lâu sẽ dẫn tới xung đột (conflict) nghiêm trọng khi ghép code chung.

- **Cách làm đúng:** Khi bạn hoàn thành xong một module nhỏ độc lập (ví dụ: xong Entity và Repository của một bảng), hãy tạo ngay một PR nhỏ để merge vào `dev`.
- Điều này giúp code của cả đội luôn được tích hợp liên tục và giảm thiểu tối đa rủi ro lỗi xung đột.

 4. PHÂN CHIA ISSUE SPRINT 2 TRÊN GITHUB

Toàn bộ công việc của Sprint 2 hiện tại được gom lại thành **4 Issue lớn** sau:

Issue ID	Tên Issue	Người phụ trách	Mô tả tóm tắt
#71	Quản trị dữ liệu nền cốt lõi	Ahug05	CRUD User, Card, Vehicle Type, Pricing Rule trên .NET Core. Cấu hình JWT Token Claims.
#72	Quản trị cấu trúc bãi đỗ xe	ToTrieuTien	CRUD Floor, Area, Slot, Gate trên .NET Core. Ràng buộc nghiệp vụ trùng lặp cấu trúc bãi.
#73	APIs đọc dữ liệu & Khung Dashboard	fong-gif	Viết APIs chỉ đọc public (info, slots bận/trống, pricing) & Dashboard trên Spring Boot.
#74	UI Đăng nhập & Quản lý dữ liệu nền	*Frontend Dev*	Giao diện Login, Route Guard bảo vệ, CRUD các trang quản trị nền bãi xe trên React.

 5. HƯỚNG DẪN KỸ THUẬT & CẤU HÌNH CỤC BỘ

 Không lưu thông tin đăng nhập trong code (Cấu hình Biến môi trường)

Để bảo mật dự án và tránh rò rỉ thông tin đăng nhập thực tế của Supabase lên GitHub, toàn bộ chuỗi kết nối Database và JWT Secret đã được cấu hình dạng **đọc qua biến môi trường**.

- **Spring Boot API** sử dụng các biến: `DB_URL` , `DB_USERNAME` , `DB_PASSWORD` , và `JWT_SECRET` .

- **.NET Core API** sử dụng các biến: `ConnectionStrings__DefaultConnection` và `Jwt__Secret`.

Hướng dẫn tạo và chạy file cấu hình môi trường cục bộ (env):

Cách 1: Sử dụng PowerShell (Khuyến dùng trên Windows)

1. Tạo một file tên là `env.ps1` ở thư mục gốc của dự án (thư mục chứa file `.gitignore`).
2. Viết nội dung cấu hình kết nối database của bạn vào file đó:

```
# --- CẤU HÌNH CHO SPRING BOOT SUPPORT API ---
$env:DB_URL="jdbc:postgresql://localhost:5432/parking_db" # Hoặc URL kết nối database
$env:DB_USERNAME="postgres"
$env:DB_PASSWORD="your_password"
$env:JWT_SECRET="DEVELOPMENT_SECRET_KEY_FOR_LOCAL_TESTING_ONLY_2026_SWP391"

# --- CẤU HÌNH CHO .NET CORE API ---
$env:ConnectionStrings__DefaultConnection="Host=localhost;Database=parking_db;User=postgres;Password=your_password"
$env:Jwt__Secret="DEVELOPMENT_SECRET_KEY_FOR_LOCAL_TESTING_ONLY_2026_SWP391"
```

3. Mỗi khi mở cửa sổ PowerShell mới để chạy code hoặc test, hãy chạy file này trước (gọi là Dot-Sourcing) để nạp biến môi trường vào terminal:

```
. .\env.ps1
```

Cách 2: Sử dụng Command Prompt (CMD)

1. Tạo một file tên là `env.bat` ở thư mục gốc của dự án.
2. Viết nội dung sau vào file:

```
:: --- CẤU HÌNH CHO SPRING BOOT SUPPORT API ---
set DB_URL=jdbc:postgresql://localhost:5432/parking_db
set DB_USERNAME=postgres
set DB_PASSWORD=your_password
set JWT_SECRET=DEVELOPMENT_SECRET_KEY_FOR_LOCAL_TESTING_ONLY_2026_SWP391
```

```
:: --- CẤU HÌNH CHO .NET CORE API ---  
set ConnectionStrings__DefaultConnection=Host=localhost;Database=parking_db;Usernam  
set Jwt__Secret=DEVELOPMENT_SECRET_KEY_FOR_LOCAL_TESTING_ONLY_2026_SWP391
```

3. Chạy file này trên cửa sổ CMD trước khi chạy ứng dụng:

```
env.bat
```

Lưu ý

⚠ **CỰC KỲ QUAN TRỌNG:** Tuyệt đối không được commit file `env.ps1` hoặc `env.bat` của bạn lên GitHub. Hai file này đã được khai báo trong `.gitignore` để Git tự động bỏ qua.

Các lệnh khởi chạy & kiểm tra lỗi cục bộ (Sau khi nạp Env)

Mỗi khi mở cửa sổ terminal mới để chạy code hoặc test, hãy chắc chắn đã **chạy file env trước** trước khi thực hiện các lệnh chạy dự án dưới đây trong cùng một terminal.

1. Phân hệ .NET Core API (Issue #71 & #72):

- **Lệnh khởi chạy (Local Server):**

```
# Bước 1: Nạp biến môi trường (ví dụ bằng PowerShell)  
.\env.ps1  
# Bước 2: Di chuyển vào thư mục API và chạy  
cd backend/ParkingBuilding.CoreApi  
dotnet run
```

API và giao diện Swagger UI sẽ chạy cục bộ tại: `http://localhost:5000/swagger` hoặc `https://localhost:5001/swagger` .

- **Lệnh kiểm tra lỗi biên dịch:**

```
cd backend/ParkingBuilding.CoreApi
dotnet build
```

2. Phân hệ Spring Boot Support API (Issue #73):

- **Lệnh khởi chạy (Local Server):**

```
# Bước 1: Nạp biến môi trường (ví dụ bằng PowerShell)
. .\env.ps1
# Bước 2: Di chuyển vào thư mục Support API và chạy
cd backend/parking-building-support-api
mvn spring-boot:run
```

API chỉ đọc và dashboard Spring Boot sẽ chạy cục bộ tại: `http://localhost:8080` .

- **Lệnh chạy các unit/integration test:**

```
cd backend/parking-building-support-api
mvn clean test
```

3. Phân hệ Frontend React/Vite (Issue #74):

(Lưu ý: Phân hệ frontend không cần nạp biến môi trường cục bộ qua file env)

- **Lệnh khởi chạy máy chủ phát triển (Dev Server):**

```
cd frontend
npm run dev
```


Giao diện React/Vite sẽ chạy cục bộ tại: `http://localhost:5173` .

- **Lệnh biên dịch thử nghiệm (Production Build):**

```
cd frontend  
npm run build
```